СОДЕРЖАНИЕ

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ 8
ВВЕДЕНИЕ 9
1 Анализ предметной области 11
1.1 Описание и функциональность существующего решения 11
1.2 Проблемы и ограничения существующего решения 14
1.3 Обзор систем оркестрации бизнес-процессов 15
1.3.1 Сравнение существующих платформ 16
1.3.2 Архитектура и ключевые концепции Temporal 17
2 Проектирование решения 20
2.1 Требования к системе 20
2.2 Архитектура и иерархия бизнес-процессов 21
2.3 Стратегия хранения промежуточных данных 23
2.4 Стратегия миграции и поэтапного развёртывания 25
2.5 Мониторинг и наблюдаемость 26
3 Реализация решения 28
3.1 Реализация бизнес-процессов и активностей 28
3.2 Механизм пакетной обработки 31
3.3 Механизм политик повторных попыток 33
3.4 Тестирование 35
4 Оценка результатов 39
4.1 Сравнение функциональности с существующим решением 39
4.2 Анализ надёжности при моделировании сбоев 39
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 40

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

ВВЕДЕНИЕ

Платформа Авто.ру по продаже автомобилей обрабатывают миллионы объявлений от тысяч дилеров ежедневно. Своевременная и корректная публикация актуальных предложений напрямую влияет на конкурентоспособность платформы и удовлетворённость пользователей. Сервис интеграции, обеспечивающий приём и обработку фидов от автодилеров, является критически важным компонентом инфраструктуры. Любой сбой в его работе ведёт к устареванию данных, потере потенциальных клиентов и снижению доверия к платформе.

Существующий сервис обработки фидов в Авто.ру построен на основе устаревшей акторной системы akka-streams, в которой выявлен дефект накопления потоков, приводящий к исчерпанию памяти и аварийному завершению приложения. Помимо этого, жёстко заданное распределение задач между экземлярами приложения, кеширование данных в оперативной памяти отдельного узла и отсутствие механизмов контрольных точек делают систему уязвимой: при сбоях или перезапусках задачи теряются и не возобновляются автоматически. Низкая наблюдаемость затрудняет диагностику инцидентов и обращений технической поддержки. Совокупность перечисленных проблем обусловливает необходимость реинжиниринга сервиса с применением современных архитектурных подходов.

Перспективным направлением для обеспечения отказоустойчивости распределённых сервисов является использование платформ долговечного исполнения. Temporal — одна из ведущих систем данного класса — предоставляет механизм оркестрации бизнес-процессов, при котором состояние выполнения сохраняется в виде журнала событий на стороне сервера. Это гарантирует автоматическое восстановление после сбоев без потери прогресса обработки, горизонтальное масштабирование путём добавления инстансов приложения, а также встроенную наблюдаемость через визуальный интерфейс и метрики. Применение Temporal позволяет описывать сложные распределённые процессы на стандартных языках программирования, сохраняя при этом строгие гарантии надёжности.

Целью данной выпускной квалификационной работы является повышение надёжности сервиса интеграции автомобильных объявлений за счёт использования системы оркестрации бизнес-процессов Temporal.

1 Анализ предметной области

В данной главе приводится описание существующего сервиса интеграции автомобильных объявлений, выявляются причины его недостаточной отказоустойчивости, а также рассматриваются современные платформы оркестрации бизнес-процессов как архитектурная альтернатива существующему решению. Особое внимание уделено платформе Temporal как инструменту, удовлетворяющему функциональным и эксплуатационным требованиям к новому сервису.

1.1 Описание и функциональность существующего решения

Существующий сервис, далее именуемый Фидпроцессор, предназначен для публикации объявлений на платформе Авто.ру на основе фидов, поставляемых автомобильными дилерами. Сервис принимает ссылку на фид, выполняет его загрузку и преобразование во внутреннюю модель платформы, после чего передаёт готовые объявления в смежный сервис публикации.

Жизненный цикл одной задачи на обработку фида включает следующие этапы: чтение записи из базы данных задач, скачивание XML-документа по указанному дилером URL, разбор документа во внутреннюю сущность, обогащение каждого объявления данными из внешних сервисов и, наконец, постановка результирующих объявлений в асинхронную очередь сервиса публикации. Целевое время от момента появления задачи в базе до начала её обработки составляет 15 минут, полное завершение обработки фида ожидается в пределах 30 минут. Совокупный поток обрабатываемых данных на момент проведения исследования составляет порядка 40 тысяч фидов и 3,5 миллионов обновлений объявлений в сутки.

Внутренняя реализация Фидпроцессора построена на основе библиотеки потоковой обработки akka-streams [1], в рамках которой описан конвейер из последовательно соединённых стадий. Первая группа стадий отвечает за подготовительную работу: стадия `enforcer` обеспечивает дедупликацию одновременных запусков обработки одной и той же задачи, стадия `tasks` производит выборку задачи из базы данных, стадия `loader` — загрузку XML-документа по сети, стадия `parser` — его разбор во внутреннюю модель

ExternalOffer. Далее следует группа конвертеров, объединённых стадией mapperFlow, которая последовательно обогащает каждое объявление дополнительной информацией: проверяет наличие ранее загруженных изображений в хранилище, дозагружает новые изображения через сервис Pica, обрабатывает панорамы экстерьера и салона автомобиля, нормализует адрес автосалона через геокодер, унифицирует параметры автомобиля относительно справочника марок и моделей, формирует комплектации и список дополнительных опций, а затем преобразует обогащённую сущность в модель ApiOfferModel.Offer. Финальная группа стадий отвечает за агрегацию счётчиков успешных и ошибочных объявлений (counterAggregator), формирование пакетных запросов (sendingGroups) и отправку их в Kafka-топик feedprocessor-autoru-to-vos (sender). Схема конвейера приведена на рисунке 1.

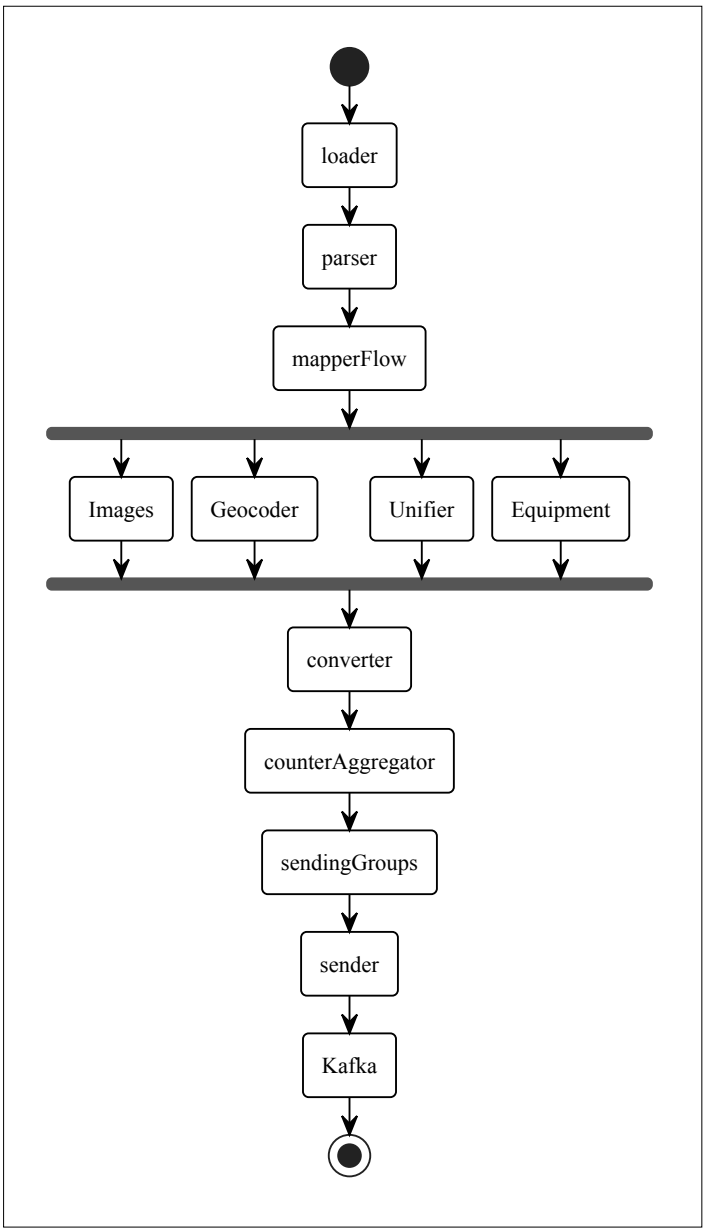


Рисунок 1 — Конвейер обработки фидов в существующем сервисе

Распределение задач между экземплярами приложения регулируется внешней системой координации Zookeeper, в которой хранится привязка отдельных дилеров к конкретным инстансам Фидпроцессора. Сервис публикации, в свою очередь, читает сообщения из Kafka-топика и сохраняет объявления в основной базе данных платформы, обеспечивая их видимость на сайте Авто.ру. Таким образом, Фидпроцессор играет роль связующего звена между внешними партнёрами и витриной объявлений, и любое нарушение его работы напрямую отражается на актуальности данных, доступных конечному пользователю.

1.2 Проблемы и ограничения существующего решения

Длительная эксплуатация Фидпроцессора в производственной среде выявила ряд системных проблем, затрагивающих как стабильность работы сервиса, так и удобство его сопровождения. Перечисленные ниже проблемы носят архитектурный характер и не могут быть устранены локальными правками без существенной переработки сервиса.

Дефект библиотеки akka-streams. В используемой версии библиотеки выявлен дефект, при котором внутренние потоки приложения постепенно накапливаются и не освобождаются после завершения обработки части задач. Указанный дефект приводит к исчерпанию доступной памяти и аварийному завершению приложения с ошибкой `OutOfMemoryError`. Исправление дефекта доступно исключительно в коммерческой редакции библиотеки, переход на которую не предусмотрен организационными и экономическими ограничениями.

Фиксированное распределенре задач. Закрепление дилеров за конкретными инстансами сервиса описано в статической конфигурации, размещённой в Zookeeper. При плановых изменениях количества инстансов, а также при проведении учений по отказоустойчивости часть задач остаётся непривязанной ни к одному из инстансов и не обрабатывается до момента ручного перераспределения. Подобная схема не поддерживает автоматическое масштабирование и приводит к проседанию пропускной способности именно в те моменты, когда устойчивость сервиса важна более всего.

Локальный кеш на уровне инстанса. Результаты обращений к внешним сервисам (унификатору, геокодеру, поставщику изображений) кешируются в оперативной памяти отдельного процесса. При каждом плановом релизе или незапланированном перезапуске инстанса накопленный кеш утрачивается, в результате чего на смежные сервисы создаётся дополнительная нагрузка по уже однажды выполненным запросам. Отсутствие межсервисного кеширования также препятствует эффективному горизонтальному масштабированию: каждый дополнительный инстанс наполняет собственный кеш независимо от остальных.

Отсутствие контрольных точек. Промежуточное состояние обработки фида хранится только в оперативной памяти конвейера и не сохраняется в надёжное хранилище. В случае падения инстанса или сетевого сбоя задача может оказаться в неопределённом состоянии: часть объявлений уже отправлена в очередь публикации, часть ещё нет, а сама задача не возобновляется автоматически. Восстановление подобных ситуаций требует ручного вмешательства разработчиков и приводит к жалобам со стороны дилеров и службы поддержки.

Низкая наблюдаемость. Существующий сервис не предоставляет удобных инструментов наблюдения за прохождением конкретного фида или объявления через этапы конвейера. Логи приложения позволяют восстановить картину произошедшего лишь частично и требуют значительных усилий при отладке ошибок обработки. Отсутствует возможность повторного запуска отдельного объявления или фида через визуальный интерфейс, что усложняет реагирование на обращения технической поддержки.

Совокупность перечисленных проблем носит системный характер: они проистекают не из ошибок реализации, а из выбранного в момент создания сервиса архитектурного подхода и устаревшей технологической базы. Их устранение локальными правками приводит лишь к временному улучшению ситуации, поэтому обоснованным представляется переход к качественно иной архитектуре, в которой отказоустойчивость, наблюдаемость и масштабируемость обеспечиваются на уровне выбранной платформы.

1.3 Обзор систем оркестрации бизнес-процессов

Под оркестрацией бизнес-процессов понимается архитектурный подход, при котором длительный многошаговый процесс описывается явным образом, а ответственность за выполнение его шагов, сохранение прогресса, обработку ошибок и согласование параллельных операций возлагается на отдельную инфраструктурную платформу. Применительно к задачам Фидпроцессора, где обработка одного фида состоит из десятков последовательных и параллельных шагов с обращением к различным сервисам и хранилищам, оркестратор позволяет вынести инфраструктурные заботы из

прикладного кода и сосредоточить внимание разработчика на бизнес-логике преобразований.

1.3.1 Сравнение существующих платформ

На рынке представлен ряд систем оркестрации, различающихся подходом к описанию процессов, моделью исполнения и требованиями к инфраструктуре. Ниже приведён обзор наиболее значимых решений, рассмотренных в рамках выбора технологии для нового сервиса.

Apache Airflow — система оркестрации задач с описанием процессов в виде направленных ациклических графов (DAG) на языке Python [2]. Платформа ориентирована на пакетную обработку данных и ETL-сценарии, выполняющиеся по расписанию. Модель исполнения Airflow слабо подходит для долгоживущих stateful-процессов, реагирующих на внешние события, а также не предоставляет встроенных средств возобновления выполнения с произвольной точки в случае сбоя.

Camunda Platform и связанная с ней система **Zeebe** — решения, реализующие стандарт BPMN 2.0 для описания бизнес-процессов [5]. Они ориентированы в первую очередь на сценарии, в которых процессы моделируются бизнес-аналитиками с использованием визуального редактора. В контексте инженерной задачи, где описание процесса разрабатывается совместно с прикладным кодом, преобладание визуального моделирования над программной нотацией создаёт дополнительные накладные расходы.

Temporal — современная платформа долговечного исполнения, выросшая из проекта Cadence и получившая существенное развитие в последующие годы [7]. Платформа предоставляет SDK для языков Java, Go, Python, TypeScript и других, описывает бизнес-процессы как обычный программный код и обеспечивает автоматическое восстановление выполнения после сбоев за счёт журналирования событий на стороне сервера.

По итогам сопоставления для решения задач реинжиниринга Фидпроцессора выбрана платформа Temporal. Среди ключевых факторов, определивших выбор, можно выделить следующие: наличие развитого Java SDK [9], совместимого с языком Scala, на котором реализован существующий

сервис; описание процессов программным кодом, что позволяет разработчикам использовать привычные средства языка для выражения нетривиальной логики; зрелость и широта сообщества пользователей Temporal в открытой экосистеме.

1.3.2 Архитектура и ключевые концепции Temporal

Кластер Temporal представляет собой распределённую систему, состоящую из нескольких слабосвязанных сервисов, каждый из которых отвечает за свой класс задач [12]. Сервис Frontend выступает точкой входа для клиентов и воркеров и реализует прикладной интерфейс взаимодействия. Сервис History отвечает за хранение и обновление журналов событий запущенных процессов. Сервис Matching управляет очередями задач, через которые воркеры получают работу. Внутренний сервис Worker обслуживает фоновые задачи самого кластера, такие как продвижение таймеров и обработка истекших таймаутов. Состояние всех перечисленных сервисов сохраняется в общем слое персистентности. Схема кластера и его взаимодействия с прикладными воркерами приведена на рисунке 2.

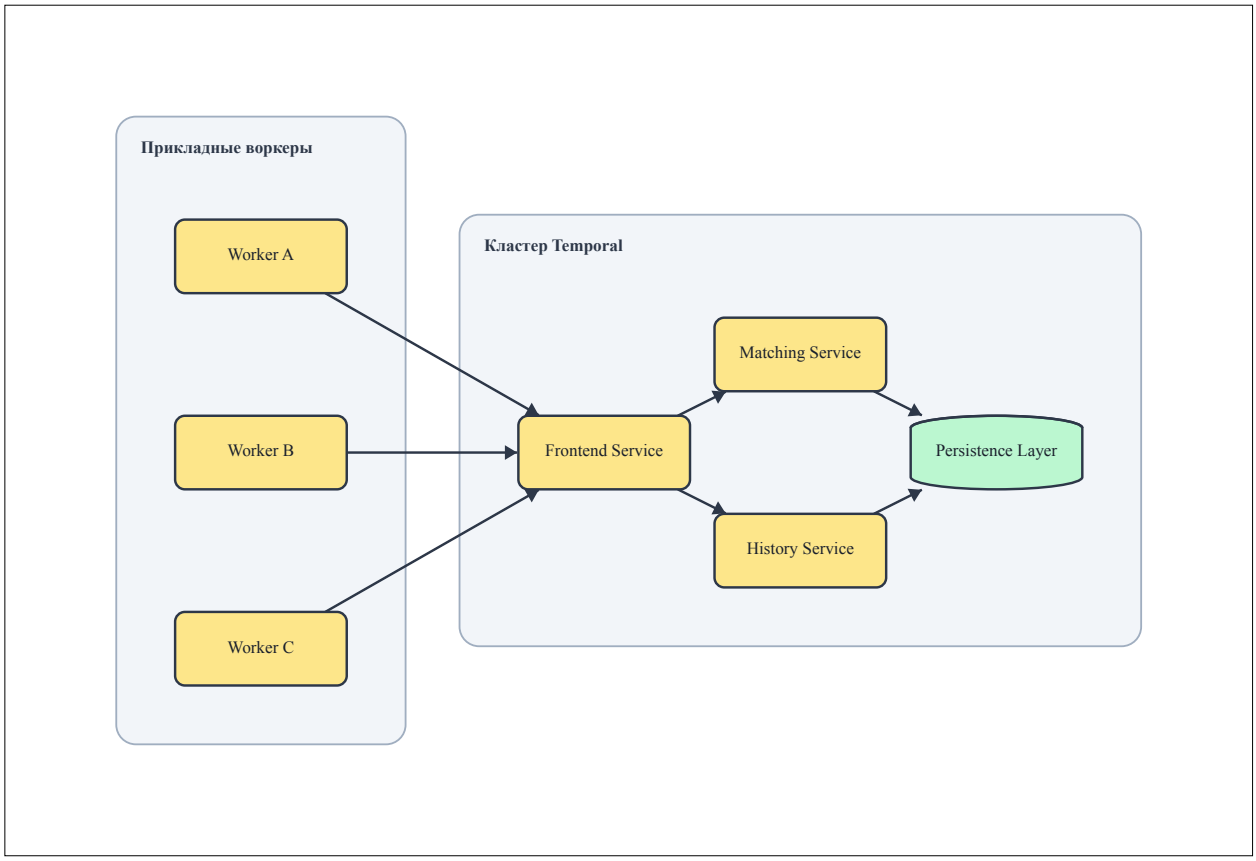


Рисунок 2 — Архитектура кластера Temporal и взаимодействие с воркерами

Описание бизнес-процессов и единиц работы в Temporal опирается на ряд понятий, которые целесообразно определить в явном виде.

Workflow — описание бизнес-процесса в виде программного кода, оркестрирующего порядок выполнения отдельных шагов. Код workflow обязан быть детерминированным относительно журнала событий: при повторном исполнении на тех же входных данных он должен производить одну и ту же последовательность вызовов.

Activity — единица работы, содержащая побочные эффекты: сетевые вызовы, обращения к базам данных, операции с файловой системой, отправка сообщений в очереди. В activity допускается недетерминированный код, при этом безопасность повторного выполнения обеспечивается за счёт идемпотентности или явных компенсаций.

Worker — процесс прикладного приложения, регистрирующий в кластере реализации workflow и activity и постоянно опрашивающий очереди задач. Добавление новых воркеров не требует изменения описания процесса и обеспечивает горизонтальное масштабирование пропускной способности.

Task Queue — логическая очередь задач, через которую сервис Matching распределяет вызовы workflow и activity между подключёнными воркерами.

History — упорядоченный журнал событий, фиксирующий каждое значимое изменение состояния workflow: запуск, планирование activity, получение результатов, истечение таймеров. Журнал является источником истины для восстановления выполнения после сбоя.

Retry Policy — настраиваемая политика повторов, задающая начальный интервал повтора, коэффициент экспоненциального наращивания задержки, максимальный интервал и предельное число попыток.

Continue-As-New — механизм, позволяющий завершить текущее исполнение workflow и начать новое с переданными аргументами, не накапливая чрезмерно длинный журнал событий в одном экземпляре процесса.

Требование детерминированности кода workflow требует отдельного рассмотрения. Восстановление выполнения после сбоя в Temporal реализовано через повторное исполнение кода workflow на основе записанного журнала событий: сервер не сохраняет произвольное состояние процесса, а воспроизводит ход его исполнения, опираясь на детерминированность вычислений. Из указанного принципа следуют практические запреты: внутри workflow недопустимо использование системного времени, генерация случайных чисел, прямые сетевые вызовы и обращения к глобальному изменяемому состоянию [14]. Перечисленные операции должны быть вынесены либо в activity, либо выполнены через специализированные API платформы — `Workflow.currentTimeMillis`, `Workflow.newRandom` или `Workflow.sideEffect`, — которые обеспечивают безопасное для повторного воспроизведения поведение.

Использование Temporal накладывает на проектируемое решение и ряд эксплуатационных ограничений, которые необходимо учитывать на этапе проектирования:

– лимит на размер аргументов workflow и activity, который не позволяет передавать через границу платформы большие двоичные данные или объёмные структуры; крупные объекты должны храниться во внешнем хранилище с передачей лишь идентификатора [6];
– необходимость контроля детерминированности кода workflow при каждом его изменении, что требует ревью кода и средств проверки совместимости со старыми историями.

Перечисленные особенности и ограничения определяют дальнейшие архитектурные решения проектируемого сервиса, рассматриваемые в следующей главе.

2 Проектирование решения

В настоящей главе формулируются требования к новому сервису, исходящие из проблем, выявленных в первой главе. Описывается декомпозиция обработки фида в иерархию бизнес-процессов на платформе Temporal, излагаются стратегии хранения промежуточных данных, поэтапной миграции с существующего сервиса и обеспечения наблюдаемости.

2.1 Требования к системе

Требования к проектируемому сервису разделены на функциональные, направленные на сохранение совместимости и пропускной способности существующего решения, и эксплуатационные, направленные на устранение проблем, описанных в разделе 1.2.

К функциональным требованиям относятся следующие:

1. Сохранение совместимости: формат фидов, модель сообщения в Kafka
2. Поддержание пропускной способности уровня существующего сервиса — порядка 40 тысяч фидов и 3,5 миллионов обновлений объявлений в сутки.
3. Соблюдение целевого времени обработки: задержка от момента появления задачи до начала её выполнения не более 15 минут, полное завершение обработки фида не более 30 минут.
4. Сохранение текущего набора стадий обогащения: дозагрузки изображений, геокодирования адресов, унификации моделей, формирования комплектаций.

К эксплуатационным требованиям относятся следующие:

1. Автоматическое восстановление выполнения после сбоев экземпляра приложения.
2. Горизонтальное масштабирование путём добавления экземпляров приложения, без ручного перераспределения задач между ними.
3. Разделяемое между всеми экземплярами хранилище результатов обращений к внешним сервисам, переживающее перезапуски приложения.
4. Возможность визуального отслеживания прохождения каждого фида и каждого объявления через стадии обработки, а также ручного перезапуска отдельных задач без переразвёртывания приложения.

Каждое эксплуатационное требование напрямую соответствует одной из проблем существующего решения и устраняется архитектурными решениями, описываемыми далее в настоящей главе.

2.2 Архитектура и иерархия бизнес-процессов

Центральным архитектурным решением является декомпозиция обработки фида в иерархию из корневого и дочерних бизнес-процессов на платформе Temporal. Подобная организация позволяет независимо отслеживать состояние обработки каждого объявления, применять локальные политики ретраев и обеспечивать раздельную наблюдаемость на уровне задачи и на уровне отдельных её элементов.

FeedWorkflow — корневой workflow, обрабатывающий за один экземпляр не более ста задач на обработку фидов. Для каждой задачи в пакете инициируется скачивание и разбор XML-документа, порождаются дочерние процессы обработки отдельных объявлений и подготовленные сообщения передаются в Kafka-топик сервиса публикации. По завершении пакета workflow вызывает Continue-As-New, передавая индекс следующей пачки новому экземпляру; оставшиеся задачи обрабатываются в следующем FeedWorkflow.

OfferWorkflow — дочерний workflow, выполняемый для каждого объявления внутри фида. Последовательно выполняет стадии обогащения — дозагрузку изображений, геокодирование адреса автосалона, унификацию марки и модели автомобиля, формирование комплектаций — и подготавливает финальную модель `ApiOfferModel.Offer` для передачи родительскому процессу.

Использование иерархии родительского и дочерних workflow обусловлено несколькими соображениями. Каждое объявление получает собственную историю событий, что позволяет диагностировать и при необходимости повторно запускать обработку отдельного объявления без повторной обработки всего фида. Ретраи стадий обогащения локализуются на уровне отдельного объявления и не блокируют обработку остальных. Ограничение пакета корневого процесса в сто фидов в сочетании с механизмом Continue-

As-New обеспечивает конечную длину истории событий FeedWorkflow и позволяет сервису работать неограниченно долго без её накопления.

Все activity, реализуемые в новом сервисе, разделены на две группы соответственно workflow, в которых они применяются:

– Activity используемые в FeedWorkflow:
  1. `loadFeed` — скачивание XML-документа по указанному дилером URL
  2. `parseFeed` — разбор документа во внутреннюю модель и сохранение списка объявлений во внешнее хранилище.
  3. `sendToKafka` — постановка пакета объявлений в Kafka-топик.
– Activity используемые в OfferWorkflow:
  1. `fetchExistingImages` — поиск ранее загруженных изображений во внутреннем хранилище
  2. `downloadImages` — загрузка новых изображений через смежный сервис
  3. `geocodeAddress` — нормализация адреса автосалона
  4. `unifyCarModel` — унификация параметров автомобиля относительно справочника марок и моделей,
  5. `prepareEquipment` — формирование комплектаций и списка дополнительных опций,
  6. `convertToApiModel` — преобразование обогащённого объявления в финальную модель.

Для каждого типа activity задаётся индивидуальная политика повторов, учитывающая характер потенциальных отказов:

– `loadFeed` — до трёх попыток с увеличенным начальным интервалом, поскольку временная недоступность URL дилера ожидаема и требует более редкого опроса;
– `downloadImages`, `geocodeAddress`, `unifyCarModel` — смежные сервисы могут быть временно недоступны, вследствие чего activity повторяется каждые 30 секунд

Схема иерархии workflow и распределения activity между task queue приведена на рисунке 3.

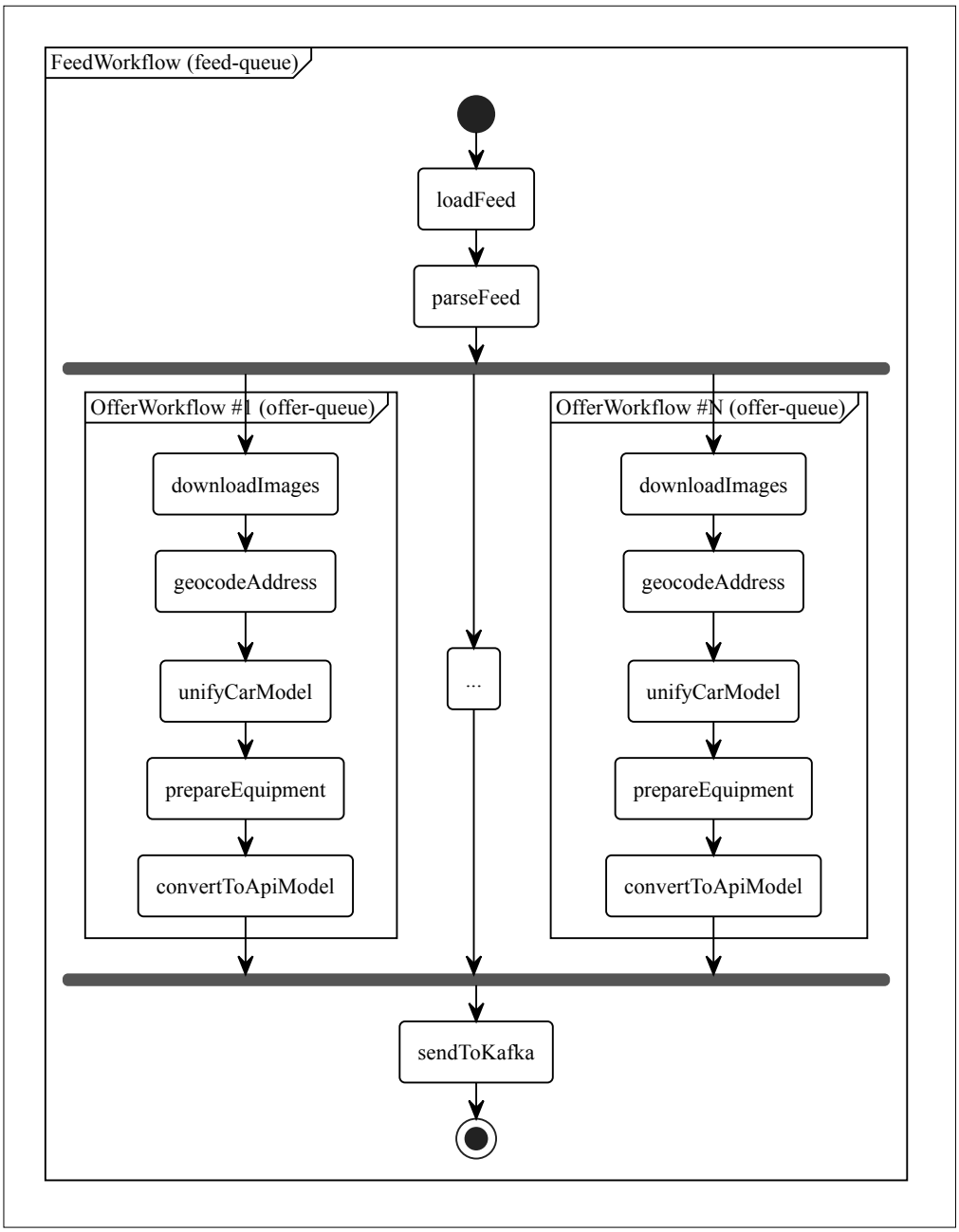


Рисунок 3 — Иерархия workflow и распределение activity между task queue

2.3 Стратегия хранения промежуточных данных

Платформа Temporal накладывает ограничение на размер аргументов и результатов workflow и activity: рекомендуемый предел составляет порядка десятков килобайт, жёсткий лимит — 5 мегабайт [6]. Прямая передача XML-документа фида, массива разобранных объявлений или ответов сервисов изображений между процессами недопустима. Решение указанной проблемы и одновременное устранение проблемы локального кеша, описанной в разделе 1.2, реализуется единым внешним хранилищем — распределённой

СУБД YDB [15], поддерживающей таблицы ключ-значение с байтовыми и текстовыми значениями.

Единое хранилище ключ-значение. Все промежуточные данные нового сервиса размещаются в нескольких таблицах YDB, объединённых общим интерфейсом ключ-значение. Через границу workflow передаются только идентификаторы строк; полезная нагрузка читается и записывается на стороне activity. Использование одной системы хранения для исходных данных и кеша упрощает эксплуатацию и снимает с разработчиков необходимость поддерживать несколько различных технологий.

К хранению больших данных применяются следующие правила:

– исходный XML-документ фида сохраняется в таблицу `raw-feeds` сразу после загрузки в activity `loadFeed`; ключ строки совпадает с идентификатором задачи на обработку и используется дочерними activity для повторного чтения документа;
– сериализованное представление одного разобранного объявления, передаваемое из FeedWorkflow в дочерний OfferWorkflow, сохраняется в таблице `parsed-offers`; ключ строки формируется детерминированно как `<feed_id>:<offer_index>`, что обеспечивает повторяемость при повторном проигрывании workflow.

Кеширование ответов внешних сервисов осуществляется по аналогичному принципу:

– ответы геокодера и унификатора кешируются в таблицах `geocoder-cache` и `unificator-cache`;
– ключ строки — хэш аргументов запроса; значение — сериализованный ответ; TTL задаётся отдельным столбцом и контролируется фоновой очисткой на стороне СУБД;
– сроки хранения выбираются исходя из частоты изменения справочников: для геокодера 30 суток, для унификатора 7 суток;
– кеш разделяемый между всеми воркерами и переживает перезапуски приложения, что устраняет дублирующие запросы к смежным сервисам при релизах нового сервиса.

2.4 Стратегия миграции и поэтапного развёртывания

Целью миграции является обратимый переход с существующего сервиса Фидпроцессор на новый сервис без остановки обработки фидов и без потери совместимости с внешними партнёрами. Поэтапная схема миграции состоит из четырёх этапов с возможностью отката на каждом из них:

1. **Теневой запуск.** Новый сервис подписан на тот же поток задач, что и существующий, и выполняет полный цикл обработки, однако результирующие сообщения не публикуются в основной Kafka-топик. Сообщения, которые сервис посчитал бы опубликованными, направляются во вспомогательный топик и сравниваются с эталоном — публикациями существующего сервиса. Расхождения логируются для последующего анализа.
2. **Канареечный запуск по дилерам.** Маршрутизация дилеров между существующим и новым сервисами регулируется флагом, размещённым в Zookeeper рядом с действующей конфигурацией привязки. Начальная доля дилеров на новом сервисе устанавливается равной одному проценту, затем по результатам контроля метрик и качества публикаций последовательно увеличивается до десяти и пятидесяти процентов.
3. **Полный перевод.** После двух недель стабильной работы при канареечной доле в пятьдесят процентов оставшиеся дилеры переводятся на новый сервис; существующий сервис переводится в режим горячего резерва и продолжает потреблять задачи, но обрабатывает их в режиме теневого запуска.
4. **Вывод из эксплуатации.** После двух недель работы нового сервиса при полной нагрузке существующий сервис останавливается, а его конфигурация в Zookeeper удаляется. Освободившиеся аппаратные ресурсы возвращаются в общий пул.

План отката предусматривает обратное изменение флага маршрутизации на любом этапе, возвращающее соответствующее подмножество дилеров на существующий сервис. Поскольку оба сервиса публикуют объявления в один Kafka-топик и используют согласованные ключи дедупликации,

переключение не приводит к потере или дублированию данных на стороне потребителя.

Сравнение результатов обработки на теневом этапе реализуется фоновой задачей, читающей оба потока сообщений и помечающей расхождения по бизнес-полям объявления. Результаты сравнения складываются в отдельную таблицу YDB и используются как критерий приёмки для перехода к канареечному этапу: переход разрешён, если доля расхождений не превышает заранее заданного значения на горизонте семи суток.

Схема распределения трафика дилеров между существующим и новым сервисами на этапах миграции приведена на рисунке 4.

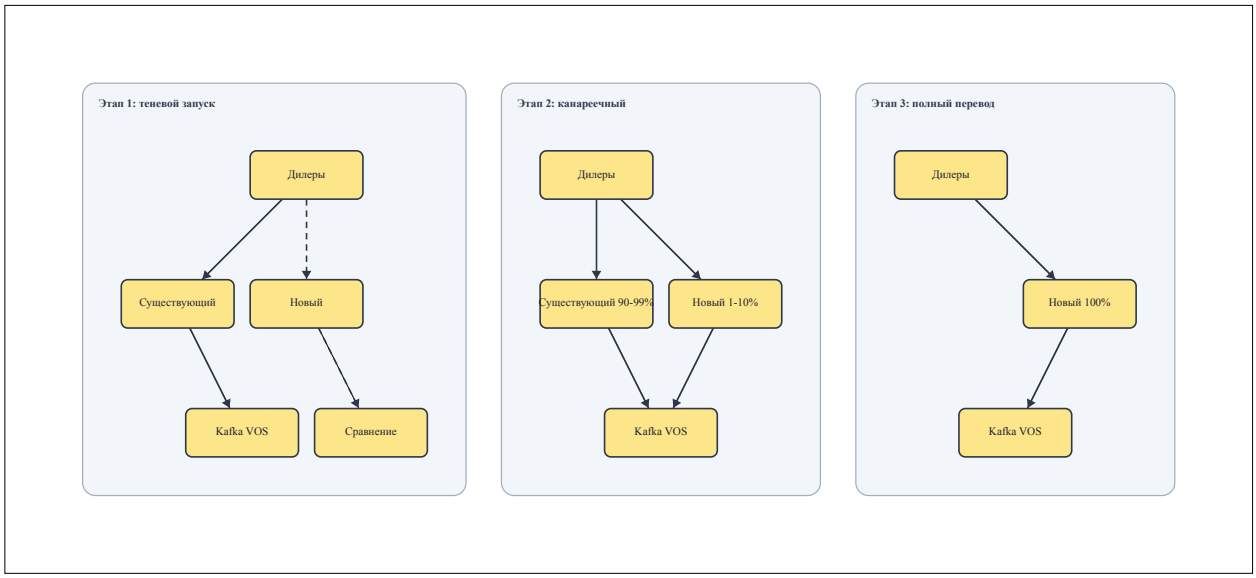


Рисунок 4 — Распределение трафика дилеров на этапах миграции

2.5 Мониторинг и наблюдаемость

Наблюдаемость нового сервиса организована на трёх уровнях: журналы выполнения workflow в кластере Temporal, метрики прикладного приложения, структурированные логи. Поверх перечисленных источников строится единая система алертинга.

Журнал выполнения workflow. Кластер Temporal предоставляет визуальный интерфейс, в котором по идентификатору workflow — равному идентификатору фида для корневого процесса или составному идентификатору объявления для дочернего — доступна полная история событий:

запуск, расписанные activity, их результаты, ретраи, исключения, истечения таймеров. Из интерфейса возможен ручной перезапуск отказавшего workflow и точечный повтор отдельной activity без переразвёртывания приложения. Указанный механизм напрямую устраняет проблему отсутствия инструментов диагностики, описанную в разделе 1.2.

Метрики. Прикладные воркеры экспортируют метрики в Prometheus, визуализация осуществляется в Grafana. К ключевым показателям относятся:

– темпы запуска и завершения workflow каждого типа в минуту;
– гистограммы длительности отдельных activity, позволяющие отслеживать деградацию смежных сервисов;
– счётчики ретраев в разрезе типа activity и причины отказа;
– глубина task queue каждой задачной очереди;
– доля workflow, превысивших целевое время обработки.

Структурированные логи. Каждая запись логов содержит поля `workflow_id`, `feed_id`, `dealer_id` и `offer_id`, что обеспечивает фильтрацию и группировку при расследовании инцидентов. Передача trace context через входные параметры workflow обеспечивает корреляцию записей нового сервиса с логами смежных сервисов через существующую систему агрегации логов.

Система алертинга формулирует пороги над описанными метриками: устойчивое превышение глубиной очереди заданного значения, доля неуспешных activity выше пяти процентов на горизонте десяти минут, длительность обработки фида свыше тридцати минут на p99-квантиле. Алерты направляются в дежурный канал поддержки сервиса.

Совокупность описанных решений по архитектуре, хранению данных, миграции и наблюдаемости образует основу для реализации нового сервиса, рассматриваемой в следующей главе.

3 Реализация решения

Настоящая глава описывает воплощение архитектурных решений главы 2 в исходном коде нового сервиса на стеке Temporal Java SDK [9] и языке Scala. Текст построен вокруг четырёх ключевых механизмов реализации — декомпозиции на workflow и activity, пакетной обработки с переходом через `continueAsNew`, политик повторных попыток и многоуровневого тестирования.

3.1 Реализация бизнес-процессов и активностей

Модель программирования Temporal Java SDK предписывает описывать workflow и activity через типизированные интерфейсы с аннотациями `@WorkflowInterface` и `@ActivityInterface` соответственно [9]. Конкретные реализации выделяются в отдельные классы с суффиксом `Impl`, что упрощает подмену реализаций в тестах и явно отделяет публичный контракт workflow от его внутренней логики. В реализуемом сервисе выделено два workflow и два набора activity, отражающих иерархию, описанную в разделе 2.2.

Корневой workflow `FeedWorkflow` принимает на вход алгебраический тип `Either` из двух вариантов: пакет идентификаторов объявлений `BatchArgs` для типового режима продолжения обработки и единичную задачу `Task` для первичного запуска по событию из очереди задач. Такая сигнатура позволяет одной и той же реализации обслуживать как первое попадание задачи на обработку, так и последующие переходы через `continueAsNew`. Интерфейс корневого workflow и сопутствующий тип аргументов приведены на листинге 1.

```
case class BatchArgs(
    uuids: Seq[UuidWithPosition],
    task: Task,
    client: Client,
    offersCount: OffersCount,
    workflowConfiguration: FeedWorkflowConfiguration)

@WorkflowInterface
trait FeedWorkflow {
  @WorkflowMethod
  def run(batchOrTask: Either[BatchArgs, Task]): Unit
}
```

Листинг 1 — Интерфейс корневого workflow `FeedWorkflow`

Тело `FeedWorkflowImpl` последовательно вызывает подготовительные activity загрузки и парсинга фида — объединённую activity `processFeedInputStream`, которая скачивает XML-документ по URL дилера, разбирает его и складывает в YDB сериализованные представления отдельных объявлений, возвращая в workflow только короткий список идентификаторов `Seq[UuidWithPosition]`. Такая компоновка позволяет соблюсти ограничение Temporal на размер аргументов и результатов activity [6], не вынося промежуточные данные через границу workflow.

После подготовки данных корневой workflow порождает дочерние процессы. Каждому идентификатору объявления соответствует отдельный дочерний workflow `OfferWorkflow`, запускаемый методом `Workflow.newChildWorkflowStub` с указанием выделенной задачной очереди `offer-queue`. Асинхронный вызов через `Async.function` возвращает объект `Promise[OfferResult]`; агрегатор `Promise.allOf` ожидает завершения всех дочерних процессов, после чего управление возвращается в корневой workflow. Соответствующий фрагмент приведён на листинге 2.

```
val promises = processNow.map { case UuidWithPosition(uuid, idx) =>
  val offerWorkflow = Workflow.newChildWorkflowStub(
    classOf[OfferWorkflow],
    offerWorkflowOptions(batchArgs.task, idx))
  Async.function(offerWorkflow.process,
    idx, batchArgs.task, batchArgs.client, uuid,
    workflowConfiguration.useExternalOfferCache)
}
Promise.allOf(promises: _*)
val entityResults = promises.map(_.get())
```

Листинг 2 — Запуск дочерних `OfferWorkflow` с ожиданием через `Promise.allOf`

Интерфейс дочернего workflow `OfferWorkflow` лаконичен: единственный метод `process` принимает короткие идентификаторы (`idx`, `task`, `client`, `externalOfferUuid` и флаг использования кеша), а полезная нагрузка считывается activity самостоятельно из YDB по uuid. Такой контракт согласуется со стратегией хранения промежуточных данных, описанной в разделе 2.3, и приведён на листинге 3.

```
@WorkflowInterface
trait OfferWorkflow {
  @WorkflowMethod
  def process(
      idx: Int,
      task: Task,
      client: Client,
      externalOfferUuid: UUID,
      useExternalOfferCache: Boolean): OfferResult
}
```

Листинг 3 — Интерфейс дочернего workflow `OfferWorkflow`

Тело `OfferWorkflowImpl` строит результирующее объявление, запуская несколько независимых activity обогащения параллельно через `Async.function`: геокодирование адреса, разбор опций, обработку панорам, загрузку изображений и унификацию модели. Получившиеся `Promise`-объекты затем сводятся в единый объект `Offer` функцией `buildOffer` после ожидания `Promise.allOf`. Каждая activity обогащения — короткая обёртка над прикладным конвертером с регистрацией результата в счётчиках метрик.

В качестве примера на листинге 4 приведена реализация activity геокодирования.

```
override def geocoding(context: Context): Result[Offer] =
  unsafeRunRetryableIO(
    registerConverterResult("geocoding") {
      geocoderConverter.convert(context)
    },
    workerMDC = WorkerMDC.fromContext(context)
  )
```

Листинг 4 — Реализация activity `geocoding`

Таким образом, workflow содержат детерминированную логику оркестрации — последовательность вызовов, ветвления, обработку результатов, — а activity инкапсулируют любое недетерминированное взаимодействие с внешним миром: обращения к YDB, http-вызовы к сервисам геокодера, унификатора и каталога, запись в Kafka. Это разделение является фундаментальным требованием Temporal для возможности replay workflow [14] и рассматривается подробнее в разделе про тестирование.

3.2 Механизм пакетной обработки

Кластер Temporal накладывает ограничение на длину истории событий workflow: рекомендуемый предел составляет порядка десятков тысяч записей, после которого начинаются предупреждения и снижение производительности [6]. При плановой нагрузке около 40 тысяч задач на обработку фидов в сутки одиночный долгоживущий workflow без переключения насытил бы историю в течение нескольких часов. Решение — обрабатывать задачи пакетами ограниченного размера и по завершении пакета переключаться на новый экземпляр workflow с пустой историей через механизм `continueAsNew`.

Алгоритм реализован в методе `processBatch` корневого workflow. Аргумент `BatchArgs` содержит упорядоченный список идентификаторов `Seq[UuidWithPosition]`; вызов разбивает его на `processNow` — первые `batchSize` элементов (по конфигурации сервиса — сто задач) — и `processLater` — оставшиеся. Пакет `processNow` параллельно обрабатывается через дочерние `OfferWorkflow` и отправляется в Kafka; если хвост

`processLater` не пуст, workflow создаёт стаб `Workflow.newContinueAsNewStub` и вызывает на нём `run` с уменьшенным аргументом, передавая управление новому экземпляру. Соответствующий фрагмент приведён на листинге 5.

```
def processBatch(batchArgs: BatchArgs): Unit = {
  val batchSize = batchArgs.workflowConfiguration.batchSize
  val (processNow, processLater) =
    batchArgs.uuids.sortBy(_.position).splitAt(batchSize.toInt)
  // ... обработка processNow через дочерние OfferWorkflow ...
  if (processLater.nonEmpty) {
    val continueAsNewStub = Workflow.newContinueAsNewStub(
      classOf[FeedWorkflow],
      ContinueAsNewOptions.newBuilder()
        .setTypedSearchAttributes(
          feedWorkflowSearchAttribute(batchArgs.task))
        .build())
    continueAsNewStub.run(
      Left(batchArgs.copy(uuids = processLater, offersCount = offersCount)))
  }
}
```

Листинг 5 — Цикл пакетной обработки с переходом через `continueAsNew`

Эффект от описанного механизма двойной. Во-первых, история событий каждого экземпляра `FeedWorkflow` ограничена событиями ста задач — запуски activity, порождения дочерних workflow, их завершения — и остаётся в пределах нескольких тысяч записей независимо от продолжительности работы сервиса. Во-вторых, восстановление после сбоя экземпляра воркера сводится к replay одной короткой истории, что укладывается в требование к времени восстановления, сформулированное в разделе 2.1. Последовательность операций при обработке одного пакета и переходе на следующий экземпляр приведена на рисунке 5.

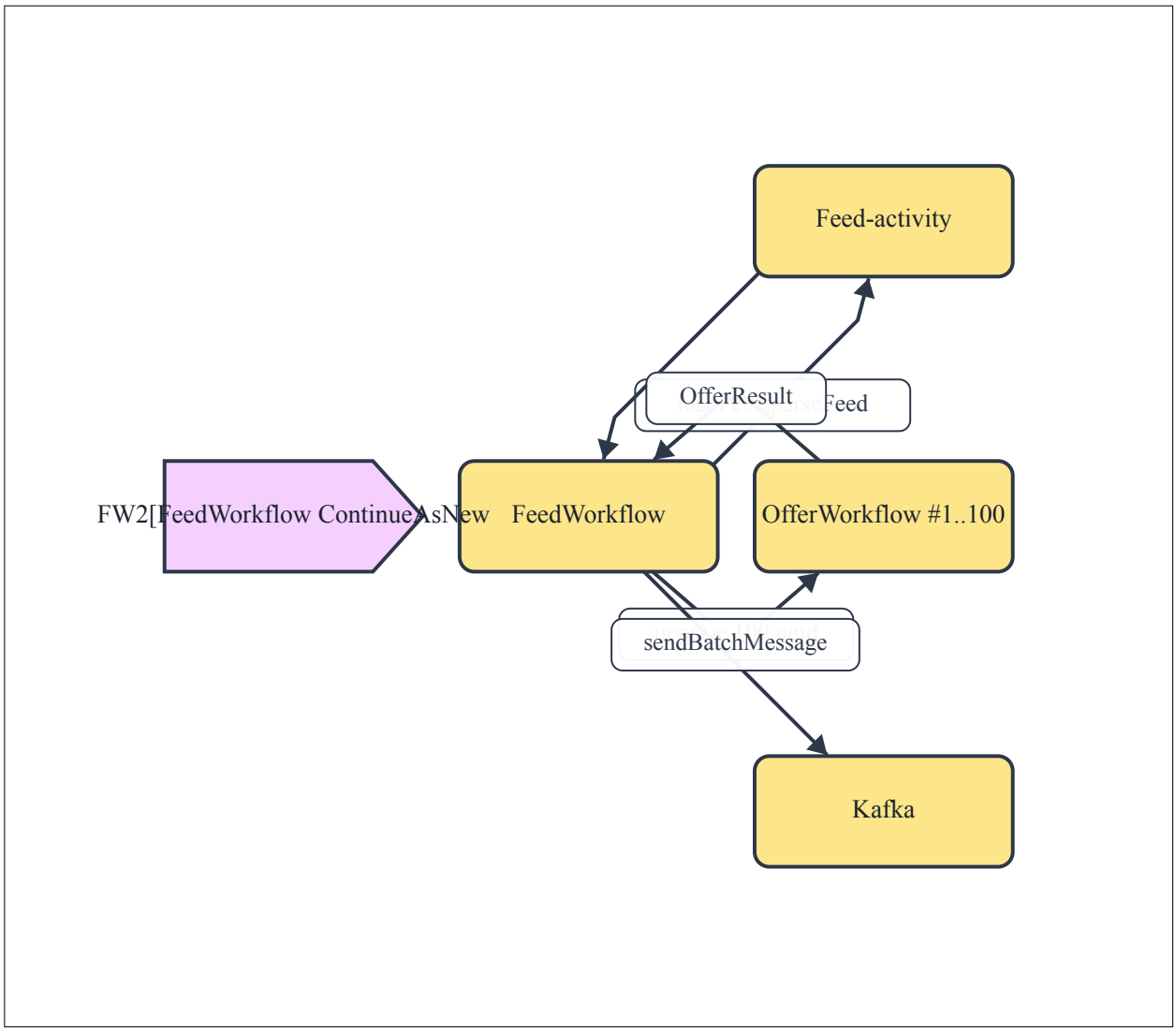


Рисунок 5 — Обработка одного пакета `FeedWorkflow` с переходом на следующий экземпляр через `continueAsNew`

3.3 Механизм политик повторных попыток

Temporal Java SDK предоставляет встроенный механизм повторных попыток activity через конфигурационный объект `RetryOptions`, описывающий начальный интервал, коэффициент экспоненциального роста, максимальный интервал и максимальное число попыток [9]. По умолчанию каждая activity, завершившаяся исключением, ретраится платформой автоматически. Реализуемая стратегия надстраивается над этим механизмом и разделяет два качественно различных типа сбоев. К **лёгким** относятся временные сетевые ошибки и таймауты внешних сервисов, которые типично проходят сами в течение секунд; к **тяжёлым** — устойчивые сбои инфраструктуры или некор-

ректные ответы смежных сервисов, которые требуют внимания дежурного оператора.

Различие реализовано во вспомогательном объекте `RetryConfiguration` через две функции-обёртки над `ApplicationFailure`, отличающиеся типом ошибки и режимом задержки. Функция `fastRetryFailure` помечает ошибку типом `network_failure` и полагается на штатную политику ретраев activity — следующая попытка выполняется через 10 секунд, далее интервал экспоненциально растёт до 10 минут. Функция `postponeRetryFailure` помечает ошибку типом `worker_failure` и устанавливает явную задержку до следующего ретрая, зависящую от номера попытки: 3, 6, 9, 12, 15 минут с потолком в 15 минут. Длинная задержка между попытками даёт дежурному время отреагировать на алерт прежде, чем activity снова обратится к внешнему сервису. Реализация обеих функций приведена на листинге 6.

```
object RetryConfiguration {
  private val NETWORK_FAILURE_TYPE = "network_failure"
  private val WORKER_FAILURE_TYPE = "worker_failure"

  def fastRetryFailure(message: String, cause: Throwable) =
    ApplicationFailure.newFailureWithCause(
      message, NETWORK_FAILURE_TYPE, cause)

  def postponeRetryFailure(message: String, cause: Throwable, attempt: Int) = {
    val maxDelay = Duration.ofMinutes(15)
    val delay = Duration.ofMinutes(3).multipliedBy(attempt)
    val methodDelay = if (delay.compareTo(maxDelay) <= 0) delay else maxDelay
    ApplicationFailure.newFailureWithCauseAndDelay(
      message, WORKER_FAILURE_TYPE, cause, methodDelay)
  }
}
```

Листинг 6 — Функции-обёртки `fastRetryFailure` и `postponeRetryFailure`

Для базовой политики ретраев каждый workflow определяет общий объект `ActivityOptions`, передаваемый при создании activity-стабов через `Workflow.newActivityStub`. На листинге 7 приведена конфигурация, используемая в `OfferWorkflowImpl`: начальный интервал 10 секунд, коэффициент удвоения, максимальный интервал 10 минут, отсутствие явного ограниче-

ния числа попыток (отсечение происходит по `startToCloseTimeout` всего workflow).

```
private val activityOptions: ActivityOptions = ActivityOptions
  .newBuilder()
  .setStartToCloseTimeout(Duration.ofMinutes(1))
  .setCancellationType(ActivityCancellationType.WAIT_CANCELLATION_COMPLETED)
  .setRetryOptions(
    RetryOptions.newBuilder()
      .setInitialInterval(Duration.ofSeconds(10))
      .setBackoffCoefficient(2d)
      .setMaximumInterval(Duration.ofMinutes(10))
      .build())
  .build()
```

Листинг 7 — Конфигурация `ActivityOptions` с `RetryOptions` в `OfferWorkflow`

Выбор между быстрой и отложенной политикой делается в реализациях activity индивидуально, в зависимости от смысла операции. Так, в `FeedActivitiesImpl` метод `getFirstTasks` оборачивает ошибку в `fastRetryFailure` — получение очереди задач не критично, и плотные ретраи допустимы, — тогда как метод `getTask` оборачивает ошибку в `postponeRetryFailure`: одна конкретная задача существенна, и её устойчивый сбой должен попадать в алерт. Activity обогащения в `OfferActivitiesImpl` — `geocoding`, `parseOptions`, `uploadImages`, `unify` — полагаются на штатную политику ретраев `ActivityOptions` и не требуют явного оборачивания. Такое разделение позволяет реагировать на устойчивые сбои не воркерами, а человеком, не нагружая внешние сервисы плотной серией повторных запросов; одновременно идемпотентность activity, обеспеченная схемой хранения раздела 2.3, гарантирует безопасность повторов в обеих политиках.

3.4 Тестирование

Тестирование Temporal-приложений организовано в три уровня, дополняющих друг друга:

1. модульные тесты конвертеров и менеджеров, инкапсулирующих бизнес-логику отдельных стадий обогащения; не отличаются от модульных

тестов любого другого Scala-сервиса и реализованы в каталоге `logic/converters/test`

2. интеграционные тесты workflow в локальном in-memory кластере `TestWorkflowEnvironment`, предоставляемом Temporal Java SDK [9]
3. replay-тесты исторических workflow на сохранение детерминизма [14]

Модульные тесты решают задачу проверки преобразований данных. Интеграционные тесты проверяют корректность оркестрации — порядок вызовов activity, ветвления по результатам, работу таймеров и кенселляции — без обращения к реальному кластеру Temporal. Третий уровень тестирования — replay-тесты — заслуживает отдельного описания, поскольку именно он закрывает риск ошибки, специфичной для платформы Temporal.

Детерминизм workflow означает, что при повторном выполнении кода workflow на зафиксированной истории событий путь исполнения должен в точности совпасть с исходным [14]. Это условие обеспечивает корректное восстановление выполнения после сбоев и переноса с одного экземпляра воркера на другой. Однако даже семантически безопасное изменение кода workflow — например, перестановка двух независимых вызовов activity — может нарушить детерминизм для уже существующих в проде долгоживущих workflow и привести к их зависанию с исключением `NonDeterministicException`. Replay-тест обнаруживает такие изменения до раскатки.

Тест устроен следующим образом: класс `WorkflowReplayerTest` подтягивает из продакшен-кластера истории нескольких реальных workflow по идентификаторам (`workflowId, runId`), после чего на локальном `TestWorkflowEnvironment` с зарегистрированными актуальными реализациями workflow вызывает метод `replayWorkflowExecution`. Любое отклонение пути исполнения от записанного в истории приводит к исключению, что фиксируется как падение теста. Ключевые фрагменты приведены на листинге 8.

```
private def makeWorker: Worker = {
  val testEnv = TestWorkflowEnvironment.newInstance(/* ... */)
  val worker = testEnv.newWorker(OFFERS_TASK_QUEUE)
  worker.registerWorkflowImplementationTypes(
    classOf[ScheduleWorkflowImpl],
    classOf[FeedWorkflowImpl],
    classOf[OfferWorkflowImpl])
  worker
}

private def replayTest(data: WorkflowData) =
  test(s"${data.workflowId}:${data.runId}") {
    for {
      client <- ZIO.service[WorkflowClient]
      worker <- ZIO.service[Worker]
      history <- ZIO.attemptBlocking {
        client.fetchHistory(data.workflowId, data.runId)
      }
      replay = Try(worker.replayWorkflowExecution(history))
    } yield assert(replay)(isSuccess.assertion)
  }
```

Листинг 8 — Регистрация workflow в `TestWorkflowEnvironment` и replay-тест на детерминизм

Набор историй для replay подбирается так, чтобы покрыть оба типа workflow и основные ветви исполнения: десять выполнений `FeedWorkflow`, десять выполнений `OfferWorkflow` и два выполнения вспомогательного `ScheduleWorkflow`, обслуживающего поступление задач из очереди. Перед раскаткой изменений CI прогоняет полный набор; падение replay-теста блокирует выкатку. Соотношение описанных слоёв реализации между собой и со слоем хранения, описанным в разделе 2.3, приведено на рисунке 6.

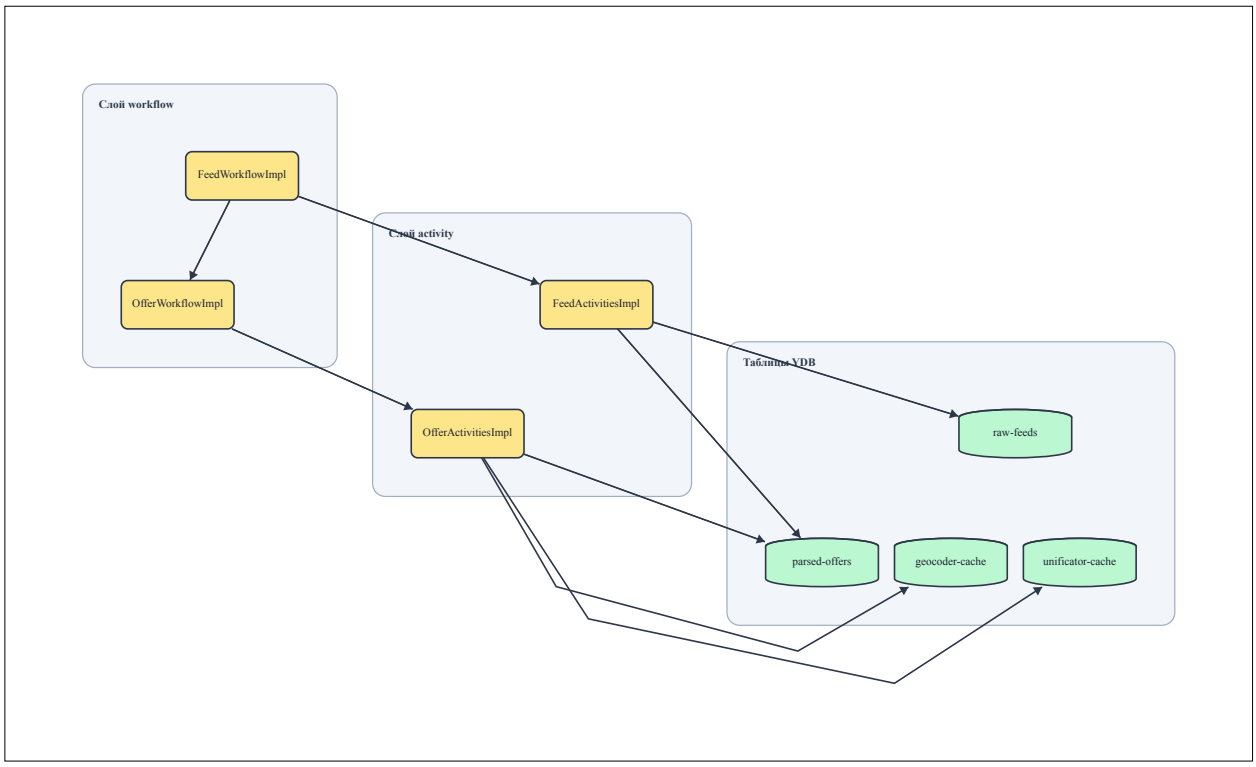


Рисунок 6 — Слои реализации сервиса и их связь с таблицами YDB

Трёхуровневая стратегия покрывает все типичные ошибки реализации Temporal-сервиса: ошибка в бизнес-логике обнаруживается модульным тестом, ошибка оркестрации — интеграционным тестом на `TestWorkflowEnvironment`, ошибка детерминизма — replay-тестом исторических workflow.

Описанная реализация воплощает все требования, сформулированные в разделе 2.1, и оставляет открытым вопрос количественной оценки результатов миграции — ему посвящена следующая глава.

4 Оценка результатов

4.1 Сравнение функциональности с существующим решением

4.2 Анализ надёжности при моделировании сбоев

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Akka Streams Documentation. – URL: https://doc.akka.io/docs/akka/current/stream/ (дата обращения: 10.05.2026). – Текст : электронный.
2. Apache Airflow. – URL: https://airflow.apache.org/ (дата обращения: 10.05.2026). – Текст : электронный.
3. AWS Step Functions | Serverless Workflow Orchestration. – URL: https://aws.amazon.com/step-functions/ (дата обращения: 10.05.2026). – Текст : электронный.
4. Cadence Workflow. – URL: https://cadenceworkflow.io/ (дата обращения: 10.05.2026). – Текст : электронный.
5. Camunda Platform 8 | The Universal Process Orchestrator. – URL: https://camunda.com/ (дата обращения: 10.05.2026). – Текст : электронный.
6. Defaults, limits, and configurable settings | Temporal Cloud. – URL: https://docs.temporal.io/cloud/limits (дата обращения: 10.05.2026). – Текст : электронный.
7. Durable Execution Solutions | Temporal Documentation. – URL: https://docs.temporal.io/ (дата обращения: 10.05.2026). – Текст : электронный.
8. Grafana Documentation. – URL: https://grafana.com/docs/grafana/latest/ (дата обращения: 11.05.2026). – Текст : электронный.
9. Java SDK Developer Guide | Temporal Platform Documentation. – URL: https://docs.temporal.io/develop/java (дата обращения: 10.05.2026). – Текст : электронный.
10. Netflix Conductor: A microservices orchestrator. – URL: https://conductor.netflix.com/ (дата обращения: 10.05.2026). – Текст : электронный.
11. Prometheus: Monitoring system & time series database. – URL: https://prometheus.io/docs/ (дата обращения: 11.05.2026). – Текст : электронный.
12. Temporal Service | Temporal Platform Documentation. – URL: https://docs.temporal.io/temporal-service (дата обращения: 10.05.2026). – Текст : электронный.

13. Temporal в Яндексе | Внутренняя документация. – URL: https://docs.yandex-team.ru/temporal (дата обращения: 10.05.2026). – Текст : электронный.
14. Workflow Execution | Temporal Platform Documentation. – URL: https://docs.temporal.io/workflows (дата обращения: 10.05.2026). – Текст : электронный.
15. YDB Documentation. – URL: https://ydb.tech/docs/ (дата обращения: 11.05.2026). – Текст : электронный.